

Proyecto 2: Divide y vencerás

Felipe Paillacar[†], Constanza Araya[†] y Guillermo Vargas[†]

[†]Universidad de Magallanes

14 de mayo de 2026

Resumen

Este proyecto aborda la implementación y el análisis experimental de algoritmos basados en la estrategia de divide y vencerás integrados a un sistema de gestión de deportistas. El estudio se centra en evaluar el rendimiento de técnicas de ordenamiento como Merge Sort (con optimización de umbral) y Quick Sort (con diversas variantes de pivote), junto con métodos de búsqueda binaria y exponencial. El trabajo integra un análisis de complejidad teórica contrastado con pruebas empíricas de tiempo de ejecución para determinar la eficiencia de cada algoritmo.

■ Índice

1	Objetivos	2
2	Introduccion	3
3	Conceptos teoricos	4
3.1	Divide y venceras	4
3.2	Complejidad de un algoritmo	4
	Conceptos clave de complejidad (notacion Big O)	
3.3	Esquema de particion de Lomuto	4
4	Algoritmos de ordenamiento	4
4.1	Merge Sort	5
4.2	Merge Sort con Insertion Sort	5
4.3	Quick Sort	5
5	Algoritmos de busqueda	7
5.1	Busqueda binaria	7
5.2	Busqueda binaria recursiva	7
	Diferencias clave en ejecucion busqueda binaria iterativa y recursiva	
5.3	Complejidad temporal busqueda binaria iterativa y recursiva	7
5.4	Busqueda binaria para rangos	8
	Complejidad temporal busqueda binaria para rangos	
5.5	Busqueda exponencial	9
	Funcionamiento de la busqueda exponencial • Complejidad temporal	
5.6	Busqueda por interpolacion	9
6	Algoritmos de seleccion	10
7	Analisis de casos: ordenamiento	11
8	Analisis de casos: busqueda	12
9	Analisis de casos: seleccion	13
10	Conclusion	14

1. Objetivos

- Implementar correctamente algoritmos utilizando la estrategia de divide y vencerás.
- Analizar empíricamente la complejidad y el rendimiento de los algoritmos implementados, evaluando además el impacto de sus distintas optimizaciones y variantes.
- Documentar adecuadamente el código desarrollado y los resultados obtenidos del análisis.

2. Introduccion

3. Conceptos teoricos

3.1. Divide y venceras

Divide y venceras es una estrategia algoritmica. Un algoritmo divide y venceras tipico resuelve un problema siguiendo tres pasos:

- **Dividir:** Descompones el problema en subproblemas mas pequeños, cada subproblema debe representar una parte del problema original, Por lo general este paso emplea un enfoque recursivo para dividir el problema hasta que no es posible crear subproblemas mas pequeños.
- **Vencer:** Resolver los sub-problemas recursivamente. Esto se conoce como el caso base, es decir el subproblema es tan pequeño que se 'resolvio solo'.
- **Combinar:** Combinar apropiadamente las soluciones de los subproblemas para obtener la solucion del problema original.

3.2. Complejidad de un algoritmo

La complejidad es una medida de que tan eficiente es el algoritmo para resolver un problema, es decir es una medida de cuanto tiempo y espacio (memoria) requiere el algoritmo para producir una solucion. La complejidad de un algoritmo se expresa utilizando la notacion **Big O**. Esta notacion proporciona una forma de comparar la complejidad de distintos algoritmos. Por ejemplo, un algoritmo con una complejidad temporal de $O(n)$ se considera más eficiente que un algoritmo con una complejidad temporal de $O(n^2)$, porque crece a un ritmo más lento a medida que aumenta el tamaño de entrada. En general, el objetivo del diseño de algoritmos es encontrar algoritmos que sean eficientes y fáciles de implementar. Esto implica un compromiso entre la complejidad del tiempo y el espacio, así como otros factores como la simplicidad (facil entendimiento) y la mantenibilidad (facilidad de actualización, corrección o mejora).

3.2.1. Conceptos clave de complejidad (notacion Big O)

Es comun confundir la notacion Big O como una medida de tiempo, pero la realidad es que la notacion Big O nos indica cuantas operaciones debe realizar un algoritmo para resolver un problema, ya que si fuera una medida de tiempo podria variar dependiendo del hardware en el que se estan realizando las pruebas. Por ello se habla del mejor y peor caso, es decir el mejor caso o caso ideal seria cuando un algoritmo tiene que hacer operaciones minimas para llegar a la solucion. Por ejemplo, en el caso de la busqueda lineal, el mejor caso es cuando el elemento que se busca esta en la primera posicion del arreglo, por lo que solo se realiza una operacion. A continuacion se muestra una comparacion de las funciones de complejidad mas comunes.

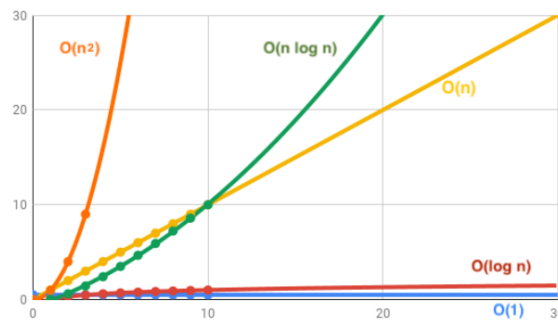


Figura 1. Comparacion de las funciones de complejidad mas comunes.

- $O(1)$: Tiempo constante. El tiempo de ejecución no cambia independiente del tamaño de la entrada.
- $O(\log n)$: Tiempo logaritmico, muy eficiente. El tiempo de ejecución crece lentamente a medida que aumenta el tamaño de la entrada.
- $O(n)$: Tiempo lineal. El tiempo de ejecución crece proporcionalmente al tamaño de la entrada.
- $O(n \log n)$: Tiempo log-lineal. Es casi una línea recta, pero con una ligera inclinación hacia arriba.
- $O(n^2)$: Tiempo cuadratico. El tiempo de ejecución crece proporcionalmente al cuadrado del tamaño de la entrada.

3.3. Esquema de particion de Lomuto

El esquema de particion de Lomuto es un algoritmo popular utilizado para dividir una matriz. Lomuto funciona seleccionando un elemento de pivote, generalmente el ultimo elemento del array, y luego dividiendolo en dos subarrays de modo que los elementos menores o iguales al pivote esten en el lado izquierdo del pivote y los elementos mayores queden a su derecha. El esquema de particion de Lomuto funciona de la siguiente manera:

- Se selecciona un elemento del array como pivote (generalmente el ultimo).
- Se recorre la lista comparando cada elemento con el pivote. Los menores se mueven a la izquierda y los mayores a la derecha.
- Al final del recorrido, se coloca el pivote en su lugar definitivo (entre las dos sublistas)
- Se repite el mismo proceso en la sublista izquierda y en la derecha hasta que todos los elementos estén ordenados.

4. Algoritmos de ordenamiento

Los algoritmos de ordenamiento son un conjunto de procedimientos que organizan los elementos de una lista o arreglo en un orden específico, ya sea ascendente o descendente. Los algoritmos de ordenamiento son importantes ya que facilitan la busqueda, la organizacion y el analisis de

datos. Son fundamentales en la eficiencia informatica ya que el desorden es costoso en terminos de tiempo y recursos. Algunas razones de su importancia incluyen:

■ Optimizacion de la busqueda

- **Busqueda lineal:** Si los datos estan desordenados, se debe revisar uno por uno hasta encontrar el elemento deseado (complejidad $O(n)$). En cambio, si los datos estan ordenados, se pueden utilizar algoritmos de busqueda mas eficientes.
- **Busqueda binaria:** Este algoritmo requiere que los datos esten ordenados para funcionar correctamente. Permite encontrar un elemento en tiempo logaritmico $O(\log n)$.

■ Organizacion y visualizacion:

 El ordenamiento permite agrupar elementos similares, lo cual es vital para:

- Eliminar duplicados: Es mas facil notar que un elemento esta duplicado si esta junto a su par.
- Priorizacion: En sistemas operativos o servidores, ordenar procesos por prioridad asegura que lo mas importante se ejecute primero.

Información

Antes de elegir un algoritmo de ordenamiento es importante hacer algunas preguntas: ¿Cuán grande son los datos que se ordenan? ¿Cuánta memoria hay disponible? ¿Los datos crecerán?

4.1. Merge Sort

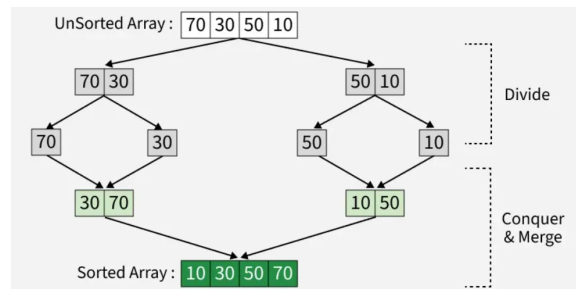


Figura 2. Merge sort no adaptativo

Merge sort es un algoritmo de ordenamiento conocido por su eficiencia y estabilidad. Utiliza la estrategia de divide y venceras para ordenar una lista o arreglo. Funciona dividiendo recursivamente el arreglo en mitades hasta que cada subarreglo contiene un solo elemento, luego combina esos subarreglos de manera ordenada para formar el arreglo final ordenado. Merge sort es un algoritmo no adaptativo, lo que significa que su rendimiento no mejora si el array esta previamente ordenado. Debido a esto su complejidad en el peor y mejor caso es $O(n \log n)$

4.2. Merge Sort con Insertion Sort

Como se menciona anteriormente en 4.1, merge sort es un algoritmo no adaptativo, lo que significa que su rendimiento no mejora si el array esta previamente ordenado. Para ello se puede optimizar utilizando insertion sort para ordenar subarreglos pequeños, ya que insertion sort es un algoritmo adaptativo, lo que significa que su rendimiento mejora si el array esta previamente ordenado, es decir Insertion sort no es 'ciego' como merge sort. Esto ayuda a que en teoria en el peor caso se mantenga la complejidad de merge sort $O(n \log n)$, pero en el mejor caso se mejora a $O(n)$, ya que insertion sort detecta que los subarreglos estan ordenados y como los subarreglos son pequeños, mejora significativamente el tiempo de ejecucion.

4.3. Quick Sort

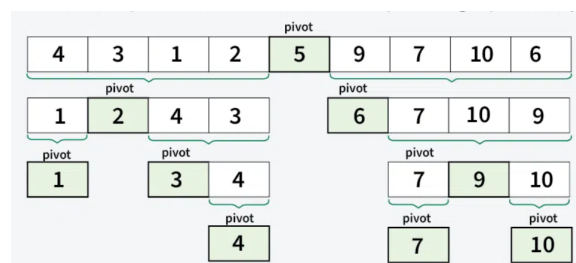


Figura 3. Quick sort

Quick sort es un algoritmo de ordenamiento que utiliza la estrategia de divide y venceras que elige un elemento como pivote y divide el array al rededor del pivote, como se nombro en [3.3](#). Este algoritmo utiliza recursividad, para ello es importante tener claro cual es el caso base, el cual es cuando el array tiene un solo elemento o esta vacio, ya que en ese caso el array ya esta ordenado. Hay diferentes formas de elegir un pivote, como por ejemplo:

- **Pivote primer elemento**
- **Pivote ultimo elemento**
- **Pivote aleatorio**
- **Pivote mediana de tres**

La eleccion del pivote es importante ya que puede afectar significativamente el rendimiento del algoritmo. Lo cual sera analizado en el transcurso de este proyecto.

5. Algoritmos de busqueda

Los algoritmos de busqueda funcionan mediante un metodo paso a paso para localizar datos especificos entre un conjunto de datos. Son métodos computacionales para localizar un elemento específico dentro de un conjunto de datos, devolviendo su posición o confirmando su existencia.

5.1. Busqueda binaria

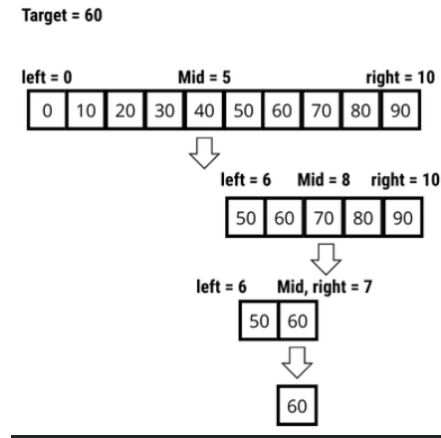


Figura 4. Busqueda binaria

La busqueda binaria es un algoritmo que se basa en el principio de 'divide y venceras'. Permite encontrar elementos en una lista ordenada mucho mas rapido que la busqueda lineal (bajo ciertas condiciones), esto se debe a que la busqueda binaria reduce el espacio de busqueda a la mitad en cada iteracion, hasta llegar a la solucion o determinar que el elemento no esta presente. Se llama binaria porque en cada paso se divide el espacio de busqueda en dos partes iguales. La busqueda binaria es mas eficiente que la busqueda lineal bajo ciertas condiciones, tales como:

- **Datos ordenados:** Esta es la condicion obligatoria para que la busqueda binaria funcione, los datos deben estar ordenados (ascendente o descendente). Si la lista esta desordenada, la busqueda binaria no funcionara correctamente y se tendra que utilizar la busqueda lineal.
- **Grandes volúmenes de datos:** La diferencia se vuelve mas significativa a medida que el tamaño de la lista aumenta. Para listas pequeñas, la busqueda lineal puede ser mas rapida.

5.2. Busqueda binaria recursiva

La busqueda binaria recursiva es una implementacion de la busqueda binaria que utiliza la recursividad para dividir el espacio de busqueda. Funciona de la siguiente manera:

- **Caso base 1:** Si el limite izquierdo supera al derecho, el elemento no esta en la lista
- **Caso base 2:** Si el elemento del medio es igual al elemento buscado, se devuelve la posicion del elemento (elemento encontrado)
- **Caso recursion**
 - Si el elemento es menor que el elemento medio, se llama a la funcion para la mitad izquierda.
 - Si el elemento es mayor que el elemento medio, se llama a la funcion para la mitad derecha.

5.2.1. Diferencias clave en ejecucion busqueda binaria iterativa y recursiva

- La version iterativa es generalmente mas eficiente para sistemas con recursos limitados, ya que no consume memoria adicional, pero visualmente el codigo puede verse mas 'sucio'
- La version recursiva consume mas memoria RAM, debido a que realiza llamadas recursivas y en cada nueva llamada se crea un nuevo marco de la pila de llamadas (stack).

5.3. Complejidad temporal busqueda binaria iterativa y recursiva

Teoricamente en el caso iterativo y recursivo se mantiene la misma complejidad temporal de $O(\log n)$, esto se deduce de la siguiente manera para el peor caso:

1. Inicio: Se tiene una lista con n elementos
2. Paso 1: Se divide la lista en dos partes iguales, es decir $\frac{n}{2}$
3. Paso 2: Se divide nuevamente la mitad seleccionada en dos partes, es decir $\frac{n}{2} \cdot \frac{1}{2} = \frac{n}{4}$
4. Paso 3: Se divide nuevamente la mitad seleccionada en dos partes, es decir $\frac{n}{4} \cdot \frac{1}{2} = \frac{n}{8}$
5. Paso k: Queda $\frac{n}{2^k}$ elementos

El algoritmo se detiene en el peor de los casos cuando solo queda un elemento por revisar, de el analisis anterior ($\frac{n}{2^k}$) podemos igualar cuando solo queda un elemento, es decir:

$$\frac{n}{2^k} = 1$$

Luego despejando k, se obtiene:

$$k = \log_2(n)$$

Por lo tanto, el numero maximo de comparaciones es proporcional al logaritmo del numero de elementos, lo que se expresa como $O(\log n)$. Luego para el mejor caso, el cual ocurre cuando el elemento que se busca esta exactamente en la mitad del array en la primera comparacion, como solo se realiza una comparacion independientemente del tamaño del array, se tiene una complejidad de $O(1)$. Cabe recalcar que todos estos analisis son teoricos, los cuales se respaldaran con pruebas empiricas de tiempo de ejecución para determinar la eficiencia de cada algoritmo.

Información

La busqueda recursiva es mas facil de entender y escribir, pero puede consumir mas memoria debido a las llamadas recursivas. La busqueda iterativa es mas eficiente en terminos de memoria, pero puede ser menos intuitiva para algunos programadores.

5.4. Busqueda binaria para rangos

La busqueda binaria para rangos es una variante del algoritmo clasico, la cual se utiliza cuando no solo se quiere saber si un elemento existe, si no que se quiere encontrar un intervalo de elementos que cumplen con una condicion especifica o son identicos.

En lugar de detenerse apenas encuentra el valor, el algoritmo sigue buscando hacia los extremos para delimitar el 'rango' donde se repite ese valor o donde se cumplen ciertas condiciones.

Se utiliza principalmente para responder dos tipos de preguntas en un arreglo ordenado:

- Contar repeticiones: ¿Cuántas veces se repite un valor específico en el arreglo?
- Encontrar limites: ¿Cuáles son los indices de todos los elementos que estan entre un rango de valores específicos?

Proceso logico

En lugar de recorrer toda la lista, el algoritmo hace dos busquedas rapidas, por ejemplo si se quiere encontrar el rango 10 – 50:

- **Busqueda del limite inferior:** Se busca el numero 10. Si el 10 no esta presente, se busca el primer numero que sea mayor que 10 (por ejemplo 11). El algoritmo devuelve ese indice (indice inicio=4).
- **Busqueda del limite superior:** Se busca el numero 50. Si el 50 no esta presente, se busca el primer numero que sea menor que 50 (por ejemplo 49). El algoritmo devuelve ese indice (indice final=50).

Una vez que se tienen ambos indices, ya se sabe exactamente donde comienza y termina dicho rango, sin tener que mirar los elementos de en medio, ya que se sabe que todos los elementos que estan entre esos indices cumplen con la condicion de estar entre 10 y 50 porque la lista ya estaba ordenada.

Nota

La busqueda binaria para rangos es especialmente util en sistemas cuando se filtran productor por precio, cuando se buscan archivos por fecha, etc.

5.4.1. Complejidad temporal busqueda binaria para rangos

Peor caso:

Para encontrar un rango (un limite inferior y superior), se realizan dos busquedas binarias independientes. Teoricamente esto se calcula como:

$$O(\log n) + O(\log n) = 2 \cdot O(\log n)$$

como en analisis de algoritmos se ignoran las constantes, se tiene una complejidad temporal de $O(\log n)$ para encontrar un rango utilizando busqueda binaria para rangos en el peor caso.

Información

En el análisis de algoritmos e ingeniería, se ignoran las constantes porque lo que interesa es el comportamiento a largo plazo cuando el volumen de datos (n) se vuelve muy grande, pero para casos pequeños, las constantes pueden tener un impacto significativo en el rendimiento real del algoritmo.

Mejor caso:

El mejor caso ocurre cuando tanto el limite inferior como el superior se encuentran en la primera comparacion de sus respectivas busquedas

- **Busqueda del limite inferior:** El valor buscado esta justo en el medio del arreglo y el elemento anterior a el es diferente
- **Busqueda del limite superior:** El valor buscado esta justo en el medio del arreglo y el elemento siguiente a el es diferente

Debido a esto es de orden $O(1)$, ya que al igual que la busqueda binaria simple, si los elementos que definen el inicio y el final del rango estan en las posiciones que el algoritmo revisa primero, el tiempo de ejecucion no depende del tamaño del arreglo

5.5. Búsqueda exponencial

El algoritmo de búsqueda exponencial se centra en un rango de un array de entrada donde asume que debe estar presente el elemento requerido, y realiza una búsqueda binaria en ese pequeño rango. Este algoritmo también se conoce como búsqueda por duplicación o búsqueda de dedos.

Este algoritmo aumenta exponencialmente en potencias de dos, esto debido a que busca el elemento dando saltos cada vez mayores, es decir, primero revisa el elemento en la posición 1, luego en la posición 2, luego en la posición 4, luego en la posición 8, y así sucesivamente, hasta que encuentra un elemento que es mayor que el elemento buscado o hasta que se sale del rango del array. Una vez que se encuentra el elemento mayor que el buscado, se realiza una búsqueda binaria en el rango identificado.

5.5.1. Funcionamiento de la búsqueda exponencial

En el algoritmo de búsqueda exponencial, el salto comienza desde el primer índice del array. Por lo tanto, comparamos manualmente el primer elemento como primer paso del algoritmo.

- **Paso 1:** Se compara el elemento en la posición 1 con el elemento buscado. Si coinciden, se devuelve la posición 1.
- **Paso 2:** Si el elemento no coincide, se compara el elemento con el siguiente índice (2,4,8,...) y así sucesivamente, hasta que se encuentra un elemento que es mayor que el elemento buscado o hasta que se sale del rango del array.
- **Paso 3:** Cuando se sabe que el elemento buscado está entre el último salto del array y el penúltimo, se aplica búsqueda binaria únicamente en ese intervalo.

5.5.2. Complejidad temporal

Mejor caso:

El mejor caso ocurre cuando el elemento buscado está en la primera posición del array `array[0]`, ya que en ese caso cae en la comprobación `if (arreglo[0] == objetivo) return 0;` como se realiza una sola comparación, la complejidad temporal en el mejor caso es $O(1)$.

Peor caso:

En el peor de los casos, tenemos que recorrer el arreglo hasta encontrar el rango donde está el dato. Sin embargo, como los saltos son cada vez más grandes (1, 2, 4, 8...), llegamos a posiciones muy lejanas en muy pocos pasos. Por eso, aunque el dato esté muy al final, el tiempo que tarda no crece de forma pesada, sino que se mantiene eficiente porque solo trabaja con la parte del arreglo donde realmente está el objetivo.

Es por esto que la complejidad está dada por $O(\log i)$, esto se debe a que hay que dar k saltos para encontrar la posición i , de la siguiente manera:

$$2^{\text{saltos}(k)} = i$$

Aplicando logaritmo en ambos lados, se obtiene:

$$k = \log_2(i)$$

Luego, una vez que se encuentra el rango donde está el elemento buscado, se realiza una búsqueda binaria en dicho rango, de ?? se sabe que la búsqueda binaria tiene una complejidad de $O(\log n)$, pero este intervalo es más pequeño que el original, es por eso que se expresa como $O(\log \frac{i}{2})$, por lo tanto, la complejidad total del algoritmo de búsqueda exponencial en el peor caso es la suma de la búsqueda del rango ($O(\log i)$) y la búsqueda binaria ($O(\log \frac{i}{2})$), lo que se expresa como:

$$O(\log i) + O(\log \frac{i}{2})$$

Por propiedades de los logaritmos:

$$O(\log \frac{i}{2}) = O(\log i - \log 2) = O(\log i) - 1$$

Reemplazando en la expresión original, se obtiene:

$$O(\log i) + O(\log i) - 1 = 2 \cdot O(\log i) - 1$$

Como se mencionó anteriormente en 5.4.1, en el análisis de algoritmos se ignoran las constantes para datos muy grandes, por lo tanto, la complejidad temporal del algoritmo de búsqueda exponencial en el peor caso es $O(\log i)$.

5.6. Búsqueda por interpolación

6. Algoritmos de seleccion

7. Analisis de casos: ordenamiento

8. Analisis de casos: busqueda

9. Analisis de casos: seleccion

10. Conclusion

```

1  /* Delete from a list */
2  /* Cell pointed to by P->Next is wiped out */
3  /* Assume that the position is legal */
4  /* Assume use of a header node */
5
6  void
7  Delete( ElementType X, List L )
8  {
9      Position P, TmpCell;
10
11     P = FindPrevious( X, L );
12
13     if( !IsLast( P, L ) ) /* Assumption of header use */
14     {                     /* X is found; delete it */
15         TmpCell = P->Next;
16         P->Next = TmpCell->Next; /* Bypass deleted cell */
17         free( TmpCell );
18     }
19 }

```

Código 1. Here is an implementation of a delete function for a singly linked list in C.([Source code](#))

■ Referencias

- [1] freeCodeCamp, *Algoritmos de búsqueda: búsqueda lineal y binaria explicadas*, jun. de 2020. dirección: <https://www.freecodecamp.org/espanol/news/algoritmos-de-busqueda-lineal-y-binaria-explicadas/>.
- [2] freeCodeCamp, *La complejidad de los algoritmos simples y las estructuras de datos en JS*, jul. de 2022. dirección: <https://www.freecodecamp.org/espanol/news/la-complejidad-de-los-algoritmos-simples-y-las-estructuras-de-datos-en-js/>.
- [3] T. S. Engineer, *Lomuto Partition Scheme*, sep. de 2023. dirección: <https://medium.com/@strivingengineer/lomuto-partition-scheme-f20eef44e76d>.
- [4] freeCodeCamp, *Significado del algoritmo divide y vencerás*, ene. de 2023. dirección: <https://www.freecodecamp.org/espanol/news/significado-del-algoritmo-divide-y-venceras/>.
- [5] E. Valley, *La complejidad de los algoritmos*, sep. de 2023. dirección: <https://europeanvalley.es/noticias/la-complejidad-de-los-algoritmos/>.
- [6] freeCodeCamp, *Explicación de la notación Big O con ejemplos de Python y JavaScript*, feb. de 2024. dirección: <https://www.freecodecamp.org/espanol/news/explicacion-de-la-notacion-big-o-con-ejemplo/>.
- [7] GeeksforGeeks, *Merge Sort*, mayo de 2024. dirección: <https://www.geeksforgeeks.org/dsa/merge-sort/>.
- [8] GeeksforGeeks, *Quick Sort Algorithm*, mayo de 2024. dirección: <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>.
- [9] StuDocu, *Búsqueda exponencial: algo básico*, feb. de 2024. dirección: <https://www.studocu.com/es-mx/document/preparatoria-9-de-la-universidad-autonoma-de-nuevo-leon/tecnologia-de-la-informacion-y-comunicacion/busqueda-exponencial-algo-basico/110361390>.
- [10] Studocu, *Algoritmo recursivo de búsqueda binaria en Java: Justificación del tamaño del problema y complejidad*, <https://www.studocu.com/es/messages/question/7340543/>, Accedido el: 14 de mayo de 2026, 2024.
- [11] Tutorialspoint, *Exponential Search*, mayo de 2024. dirección: https://www.tutorialspoint.com/data_structures_algorithms/exponential_search.htm.